Python Engineering

Class 05 - Lists & Tuples

Finding maxima: normal for loops, generator expressions, and lambda

Support note for **week_02_data_structures -> class_05_lists_tuples**. We start from a trajectory stored as a list of tuples ordered as `(t, h, v)`.

1. The goal

```
trajectory = [
    (0.0, 0.0, 50.0),
    (0.1, 5.0, 49.019),
    (0.2, 9.9019, 48.038),
    ...
]
```

We want the maximum altitude. Because altitude is stored at index 1, `state[1]` represents h.

2. Long version: a normal for loop

```
heights = []

for state in trajectory:
    heights.append(state[1])

max_h = max(heights)
```

This is usually the easiest version to visualize: iterate over every state, extract its altitude, store it, and finally apply `max()`.

Yes: this version is completely correct. The compact expression shown later does not perform a fundamentally different algorithm.

3. You can even compute the maximum manually

```
max_h = trajectory[0][1]

for state in trajectory[1:]:
    if state[1] > max_h:
        max_h = state[1]
```

Here we do not even use `max()`. We implement the algorithm ourselves: keep the best value found so far and update it when a larger one appears.

4. List comprehension: a one-line for that creates a list

```
heights = [state[1] for state in trajectory]
max_h = max(heights)
```

The bracketed expression is called a **list comprehension**. It is a compact form of the previous for loop. Read it as: for every state in trajectory, produce state[1].

```
[state[1] for state in trajectory]
 ^value      ^loop
 produced
```

5. Generator expression inside max()

```
max_h = max(state[1] for state in trajectory)
```

There are no square brackets here. The expression inside max() is a **generator expression**. It also iterates over trajectory and produces one altitude at a time, but it does not first build a complete list of all altitudes.

Conceptually, `max()` receives 0.0, then 5.0, then 9.9019, and so on, keeping the largest value. This can save memory for large data sets.

6. List comprehension vs generator expression

Form	Builds list?	Typical use
[state[1] for state in trajectory]	Yes	You want to reuse all altitude values
(state[1] for state in trajectory)	No	You consume the values only once
max(state[1] for state in trajectory)	No	You only need the maximum

7. Maximum altitude vs complete maximum state

The generator-expression version returns only the maximum altitude:

```
max_h = max(state[1] for state in trajectory)
```

In engineering we often also want to know **when** it occurred and what the velocity was. Then we want the complete tuple.

8. key= and lambda

```
max_state = max(
    trajectory,
    key=lambda state: state[1]
)
```

`max()` now receives complete tuples, while `key=` tells it what criterion to use for comparing them.

`lambda state: state[1]` is a small anonymous function. It receives a state and returns its altitude. max() uses that altitude only as the comparison key, but returns the complete state.

```
state = (t, h, v)
          |
lambda state: state[1]
               |
               h

max() compares h values
but returns the whole (t, h, v)
```

9. Unpacking the maximum state

```
t_max, h_max, v_at_max = max_state

print(t_max)
print(h_max)
print(v_at_max)
```

Physically, for a vertical launch we expect `v_at_max` to be close to zero. With a discrete dt it does not need to be exactly zero.

10. The same thing without lambda: normal for loop

```
max_state = trajectory[0]

for state in trajectory[1:]:
    if state[1] > max_state[1]:
        max_state = state

t_max, h_max, v_at_max = max_state
```

This performs exactly the same idea as `max(..., key=lambda ...)`, but explicitly. For learning purposes, writing this version at least once is excellent.

11. Which approach should I use?

Need	Recommended approach
Learn the algorithm	normal for loop
Keep all altitude values	list comprehension
Only need the maximum value	generator expression + max()
Need the complete state at maximum	max(..., key=lambda ...)

12. Empty trajectory

If trajectory is empty, `max()` has nothing to compare and raises an error. Robust code can check the list first or use a default value.

```
max_state = max(
    trajectory,
    key=lambda state: state[1],
    default=None
)

if max_state is not None:
    ...
```

Learning Tracker

Concept	Status
Normal for loop to find a maximum	Learned
List comprehension	Introduced
Generator expression	Introduced
max() over a generator	Introduced
key= in max()	Introduced
lambda	Introduced
Tuple unpacking	Reinforced

Remember: the compact form does not replace the algorithm; it only expresses it more declaratively. If you understand the normal for loop, you understand what the one-line expression is doing.